Authored by : Arpit Raj , Lnmiit jaipur

DATABASE AUTO-INCREMENT NOTES

Specify Starting Value

```
ALTER TABLE Students
AUTO_INCREMENT = 1000;
```

Next insert `1000`

SERIAL (PostgreSQL)

PostgreSQL historically uses

`SERIAL`

Example

```
CREATE TABLE Students(

    StudentID SERIAL PRIMARY KEY,

    Name TEXT
);
```

> Looks like a datatype.
> Actually it expands into
>
> `Integer` + `Sequence` + `Default NEXTVAL()`
>
> Internally it creates:
> - an integer column,
> - a sequence object,
> - a default that fetches the next sequence value.

BIGSERIAL

Same idea.
But

`BIGINT`

instead of

`INT`

IDENTITY (SQL Server)

SQL Server uses

`IDENTITY`

Example

```
CREATE TABLE Students(

    StudentID INT
    IDENTITY(1,1)
    PRIMARY KEY,

    Name VARCHAR(100)
);
```

> **Meaning**
> Start = 1
> Increment = 1

MySQL

MySQL's AUTO_INCREMENT is managed internally by the storage engine (such as InnoDB). You don't interact with a separate sequence object the way you do in PostgreSQL

MySQL LAST_INSERT_ID()

```
INSERT INTO Students(Name)VALUES('Aadya');
SELECT LAST_INSERT_ID();
```

Returns `1`

It is connection/session-specific, making it safe even when many users are inserting simultaneously

Resetting Counters

Current `100`

Need `1`

MySQL

```
ALTER TABLE Students
AUTO_INCREMENT = 1;
```

> **Important:**
> If rows already exist with IDs greater than or equal to the requested value, MySQL will not reuse those IDs. The next generated value will still be greater than the current maximum.

Can AUTO_INCREMENT guarantee consecutive IDs?

No.
It guarantees unique IDs, not gap-free IDs.

Rollback , DeleteRows